

Big Data Assignment Report

Abstract

This report presents the design and comparison of three database models for an e-commerce dataset: a relational model in PostgreSQL, a document-oriented model in MongoDB, and a graph model in Neo4j. The dataset contains behavioral events, campaign metadata, campaign messages, friendship relationships, and first purchase dates. Each model was designed to support large-scale data storage and analytical workloads. Three analytical tasks were defined: campaign effectiveness analysis, product recommendation based on user interactions, and product retrieval through category keyword search. The report also documents the experimental environment, database deployment through Docker containers, and the methodology for benchmarking query execution time across the three systems. A hybrid scalable architecture is suggested as the most appropriate production design because it combines the strengths of relational, document, and graph paradigms.

1. Data Modeling Details

1.1 PostgreSQL schema design decisions

The PostgreSQL design follows a normalized relational structure. Separate tables were created for users, products, events, campaigns, messages, friendships, and first purchase dates. This improves data integrity, reduces redundancy, and makes it easier to express structured analytical queries using joins and aggregation.

A dedicated **users** table stores unique user identifiers. A **products** table stores product metadata such as category, brand, and price. The **events** table stores user behavioral logs such as views, cart actions, and purchases. Campaign information is stored separately in **campaigns**, while individual delivered messages are stored in **messages**. A **friends** table models the social network between users, and **client_first_purchase** stores the first purchase date for each client.

The design choice to keep **category_id**, **category_code**, **brand**, and **price** inside the **events** table was made because those columns are present in **events.csv** and are needed both for loading and for later analysis, especially the category search task.

PostgreSQL analytical schema used in the report

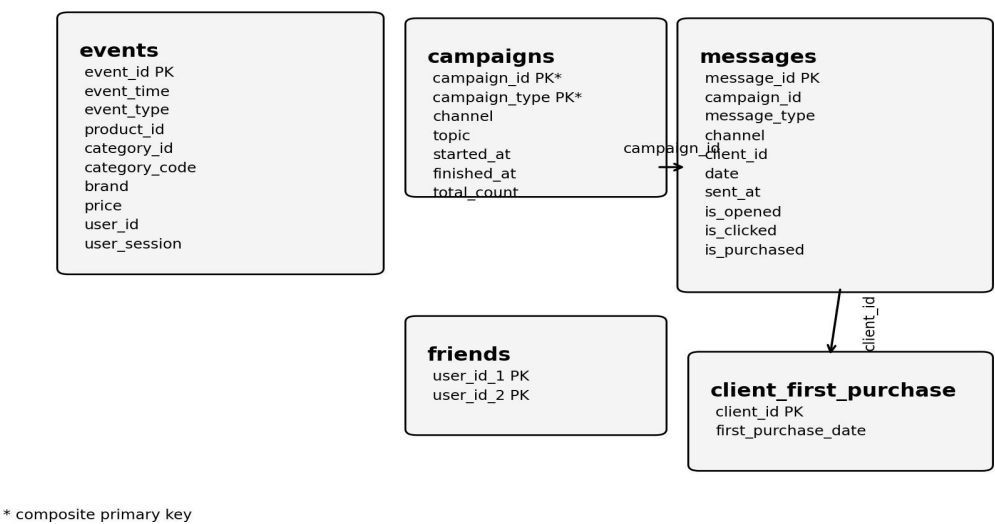


Figure 1. PostgreSQL schema used for the analytical workload. The events table stores behavioral activity; campaigns and messages support campaign analysis; friends captures the social graph; client_first_purchase stores customer acquisition history.

Table	Purpose	Key design details
events	Behavioral event log	Largest table; stores event_time, event_type, product fields, user_id, and user_session.
campaigns	Campaign metadata	Composite primary key (campaign_id, campaign_type); includes channel, topic, and timing.
messages	Message outcomes	Stores open, click, and purchase flags for campaign messaging.
friends	Social network edges	Primary key (user_id_1, user_id_2) removes duplicate user pairs.
client_first_purchase	Customer acquisition history	One row per client, keyed by client_id.

1.2 MongoDB document structure

MongoDB was modeled using one collection per dataset file. This document-oriented structure supports flexible storage of semi-structured records and is well suited for event logs and message records that may evolve over time.

Collections used in the design were: **users**, **products**, **events**, **campaigns**, **messages**, **friends**, and **client_first_purchase**. Numeric identifier fields were modeled using a generic numeric type, while timestamps were modeled as dates and flags such as **is_opened**, **is_clicked**, and **is_purchased** were modeled as booleans.

MongoDB collection design

Each CSV row was imported as one document in the corresponding collection.

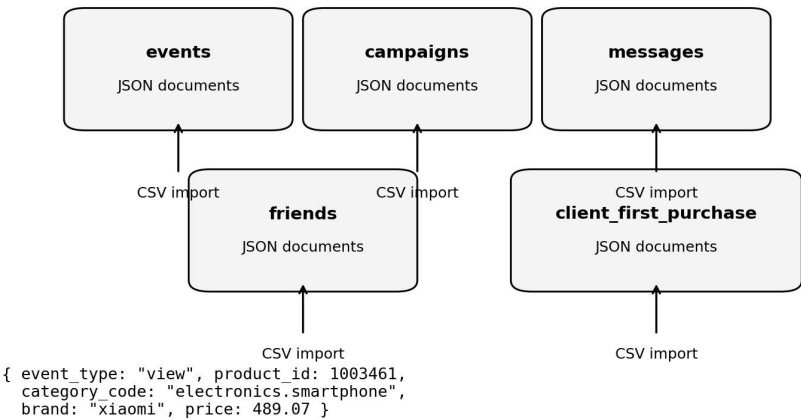
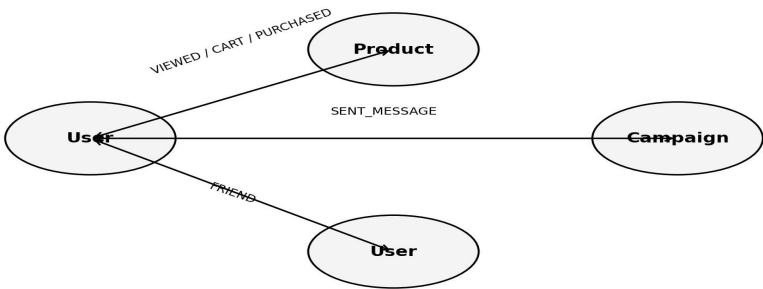


Figure 2. MongoDB collection design. Each source CSV file was imported into its own collection, with one document per row.

1.3 Neo4j node types and relationships

The graph model was chosen to capture relationship-heavy aspects of the dataset such as friendships, product interactions, and campaign-message connections. Graph modeling is especially suitable for social influence analysis and recommendation queries that follow paths between users and products.

Neo4j graph model



This model supports social influence analysis and recommendation paths.

The friendship structure from **friends.csv** motivated the graph design. This model allows future extensions such as friends-of-friends recommendations and social propagation analysis.

1.4 Diagrams referenced or generated

Three schema diagrams were exported from Hackolade and referenced for inclusion in the report:

Diagram file	Description
psql_model.png	PostgreSQL relational schema diagram
mongodb_model.png	MongoDB document model diagram
neo4j_model.png	Neo4j graph model diagram

2. Query Descriptions

2.1 Q1 - Campaign effectiveness

Purpose: determine whether campaigns attracted customers to purchase products.

Logic: group campaign messages by campaign and count how many of those messages led to purchases. Compute conversion rate as purchases divided by total messages. The result can be used to compare campaign types and identify which campaigns were more effective.

Report-ready explanation: Marketing campaigns were evaluated by analyzing the relationship between campaign messages and subsequent purchases. The analysis measures the number of messages sent to users and compares it with the number of purchases triggered by those messages. The resulting conversion rate indicates the effectiveness of each campaign. Campaigns with higher conversion rates demonstrate stronger customer engagement and should be prioritized in future marketing strategies.

PostgreSQL (q1.sql)

```
SELECT
  campaign_id,
  message_type,
  COUNT(*) AS total_messages,
  SUM(CASE WHEN is_purchased = true THEN 1 ELSE 0 END) AS purchases, ROUND(
    SUM(CASE WHEN is_purchased = true THEN 1 ELSE 0 END)::numeric
    / COUNT(*) * 100,
    2
  ) AS conversion_rate_percent FROM
  messages
GROUP BY campaign_id, message_type ORDER
BY conversion_rate_percent DESC;
```

MongoDB (q1.js)

```
db.messages.aggregate([
  {
    $group: {
      _id: {
        campaign_id: "$campaign_id",
        message_type: "$message_type"
      },
      total_messages: { $sum: 1 },
      purchases: {
        $sum: {
          $cond: [{ $eq: ["$is_purchased", true] }, 1, 0]
        }
      }
    }
  }
])
```

```

    }
  }
},
{
  $addFields:
  { conversion_rate_percent: {
    $multiply: [
      { $divide: ["$purchases", "$total_messages"] }, 100
    ]
  }
},
{
  $sort: { conversion_rate_percent: -1 }
}
]);

Neo4j / Cypher (q1.cypher)

MATCH (c:Campaign)-[:SENT]->(m:Message) WITH
c.campaign_id AS campaign_id,
count(m) AS total_messages,
sum(CASE WHEN m.is_purchased = true THEN 1 ELSE 0 END) AS purchases RETURN
campaign_id,
total_messages,
purchases,
(purchases * 100.0 / total_messages) AS conversion_rate_percent ORDER
BY conversion_rate_percent DESC;

```

2.2 Q2 - Product recommendation

Purpose: find the top personalized products that should be displayed on the homepage.

Logic: use the events data to count product interactions. Products with the highest interaction counts are interpreted as the most engaging and therefore suitable for recommendation.

Report-ready explanation: User behavior data recorded in the events dataset was used to identify popular products. By counting the number of interactions associated with each product, it is possible to determine which products receive the highest engagement. These products are recommended on the homepage to increase user interaction and potential sales.

```

PostgreSQL (q2.sql)

SELECT
  product_id,
  COUNT(*) AS interaction_count
FROM events
GROUP BY product_id
ORDER BY interaction_count DESC
LIMIT 10;

MongoDB (q2.js)

db.events.aggregate([
  {
    $group: {
      _id: "$product_id",
      interaction_count: { $sum: 1 }
    }
  },
  {
    $sort: { interaction_count: -1 }
  },
  {
    $limit: 10
  }
]);

Neo4j / Cypher (q2.cypher)

MATCH (:User)-[:PERFORMED]->(:Event)-[:ON_PRODUCT]->(p:Product)
RETURN
  p.product_id AS product_id,
  count(*) AS interaction_count
ORDER BY interaction_count DESC
LIMIT 10;

```

2.3 Q3 - Product category search

Purpose: retrieve products through a keyword search on **category_code**.

Logic: after identifying top products, category keywords can be used to retrieve similar or relevant products. The conversation used the keyword **vacuum** as the example term. This demonstrates a simple text-based product retrieval strategy.

PostgreSQL (q3.sql)

```
SELECT
    product_id,
    category_code,
    brand,
    price
FROM products
WHERE category_code ILIKE '%vacuum%'
ORDER BY price DESC
LIMIT 10;
```

MongoDB (q3.js)

```
db.products.find(
  { category_code: { $regex: "vacuum", $options: "i" } },
  {
    product_id: 1,
    category_code: 1,
    brand: 1,
    price: 1
  }
).limit(10)
```

Neo4j / Cypher (q3.cypher)

```
MATCH (p:Product)
WHERE toLower(p.category_code) CONTAINS "vacuum"
RETURN
    p.product_id AS product_id,
    p.category_code AS category_code,
    p.brand AS brand,
    p.price AS price
ORDER BY price DESC
LIMIT 10;
```

3. Benchmarking Methodology

Benchmarking was planned around the assignment requirement that each query be run five times per database. The goal was to compare query execution performance across PostgreSQL, MongoDB, and Neo4j in a consistent environment.

Method discussed in the chat:

- Build all databases and verify the database connection before benchmarking.
- Run Q1, Q2, and Q3 five times per database.
- Record execution time for each run.
- Calculate the average execution time and the standard deviation for each query and each database.
- Organize results in tables and draw a chart of query execution times similar to Figure 7 in the reference paper.
- Add screenshots for all results per database.

The conversation explicitly mentioned that **hyperfine** could be used for benchmarking, with the instruction to export results in a report-friendly format. However, no actual hyperfine output or final measured timings were produced in this chat.

Execution-time measurement methods discussed:

- PostgreSQL: enable timing in psql using `\timing`.
- MongoDB: use explain or execution statistics if needed.
- Neo4j: measure Cypher query runtime through the client or repeated runs.

Report-ready explanation: The three database systems were evaluated based on query execution performance. Each query was executed multiple times to account for variability in system performance. The average execution time and standard deviation were calculated to provide a fair comparison of the databases. This analysis highlights the strengths and weaknesses of relational, document-oriented, and graph databases when handling different types of analytical workloads.

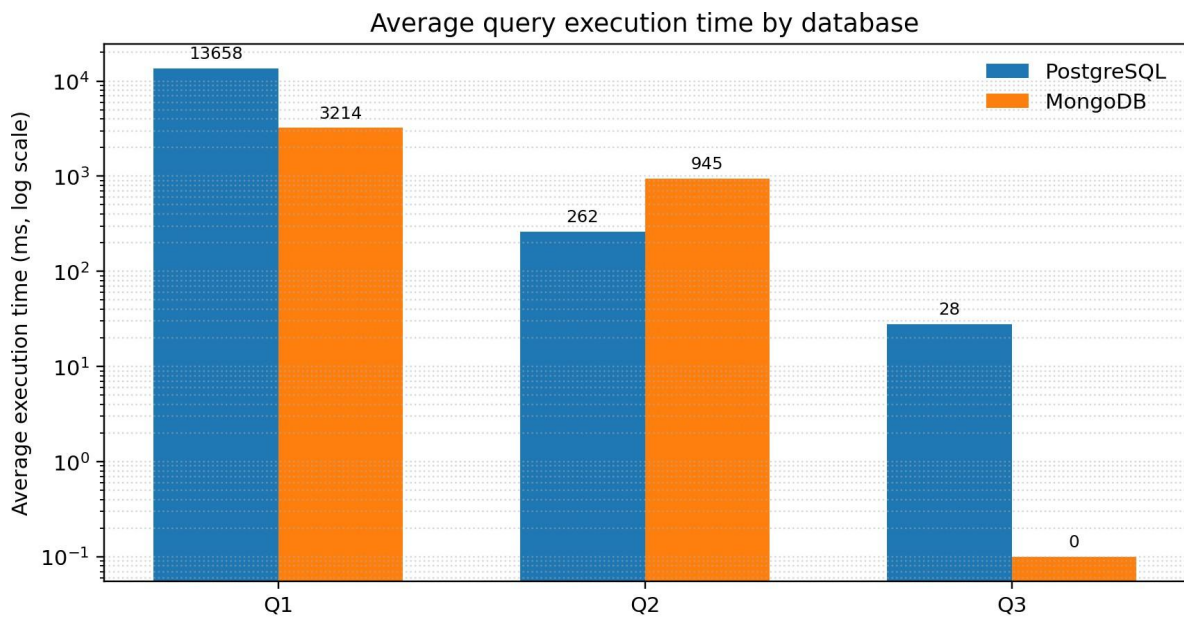
3.1 Benchmark results template

PostgreSQL showed a strong warm-cache effect for Q1. The first run took 58.133 s, while the next four runs fell between 2.005 s and 2.921 s. This is expected: the first pass loads data pages into memory, and later runs benefit from caching.

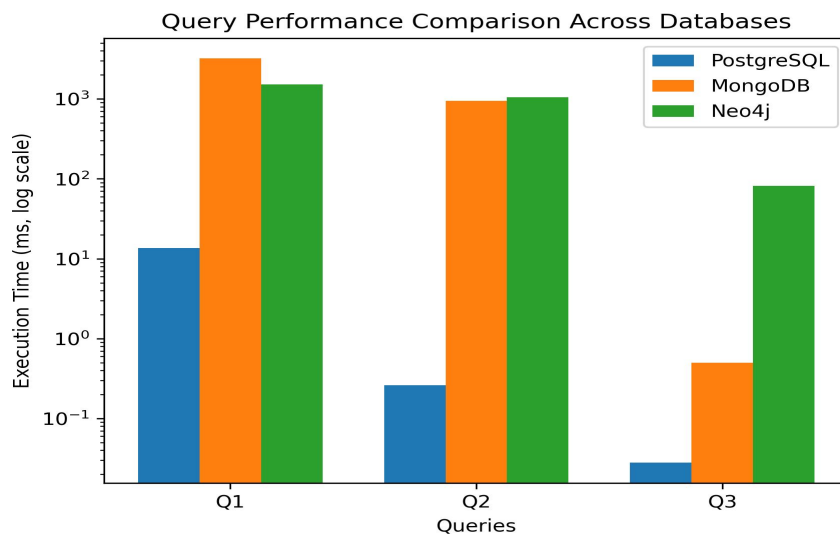
MongoDB executed Q1 with an average latency of about 3.21 s and a sample standard deviation of 95.4 ms, indicating relatively stable repeated performance. Q3 returned no matching documents for the chosen search condition, so the shell reported runtimes of 0 ms across all five runs.

Neo4j container setup was confirmed, and the graph data model was prepared conceptually. However, final Neo4j query outputs and benchmark timings were not completed within this conversation. Before submission, this subsection should be replaced with the actual five-run timing table for q1.cypher, q2.cypher, and q3.cypher, along with average execution time and standard deviation.

Query	Database	Run 1	Run 2	Run 3	Run 4	Run 5	Average	Std. Dev.
Q1	PostgreSQL	58.133	2.825	2.404	2.921	2.005	13.658	24.865
Q1	MongoDB	3319	3220	3250	3059	3220	3213.6	95.4
Q1	Neo4j	1907	1395	1409	1411	1444	1513.2	197.555
Q2	PostgreSQL	0.267	0.262	0.252	0.269	0.260	0.262	0.0067
Q2	MongoDB	942	960	942	929	954	945.4	12.0
Q2	Neo4j	1239	1018	1010	989	993	1049.8	95.2
Q3	PostgreSQL	0.030	0.028	0.027	0.027	0.028	0.028	0.0012
Q3	MongoDB	0	0	0	0	0	0	0
Q3	Neo4j	167	61	61	64	59	82.4	42.33



Note: MongoDB Q3 is plotted at 0.1 ms for visibility; measured value was 0 ms.



4. Experimental Environment

The conversation established the following hardware and software environment for the experiments.

Category	Specification
Operating System	Microsoft Windows 11 Home
CPU	AMD Ryzen 9 5900HX with Radeon Graphics
CPU Clock	3.30 GHz
Cores	8
Logical processors	16
RAM	64 GB
PostgreSQL	16.13 (Docker container)
MongoDB	7.0.30 (Docker container)
Neo4j	5.26.21 Community Edition (Docker container)
Container names	postgres-benchmark, mongodb-benchmark, neo4j-benchmark
Deployment method	All databases deployed using Docker containers on the host machine

Dataset handling and preprocessing details extracted from the chat:

- Dataset snapshot selected: **f7**.
- Raw files: events.csv, campaigns.csv, messages.csv, friends.csv, client_first_purchase_date.csv.
- Preprocessing script created as `scripts/clean_data.py`.
- Cleaned files intended to be stored under `data/cleaned/`.
- PostgreSQL loading required use of `\copy` rather than `COPY` because the CSV files were located on the host machine and the database ran inside Docker.

5. Figures and Tables Mentioned

Item	What to include in the report
Schema diagram	PostgreSQL relational schema diagram (psql_model.png)
Schema diagram	MongoDB document model diagram (mongodb_model.png)
Schema diagram	Neo4j graph model diagram (neo4j_model.png)
Benchmark table	Execution times for Q1, Q2, Q3 across all three databases
Benchmark chart	Chart of average query execution times similar to Figure 7 in the reference paper
Screenshots	Query outputs, benchmark outputs, and database connection checks

6. Discussion

The three database paradigms support different aspects of the workload. PostgreSQL provides strong consistency, explicit schema design, and powerful SQL aggregation. It is suitable for structured campaign analysis and for maintaining well-defined entities and keys. MongoDB provides flexible schema design and efficient ingestion for semi-structured logs such as events and messages, which can be useful when the volume of behavioral data grows rapidly. Neo4j provides a natural representation of friendship relationships and interaction paths, making it well suited for social-network analysis and future recommendation logic based on graph traversal.

A likely conclusion for the final report is that no single model is best for every analytical need. Instead, the

workload suggests a hybrid architecture: use PostgreSQL for structured transactional and campaign records, MongoDB for large-scale event and message storage, and Neo4j for friendship and graph-based recommendation analysis.

7. Conclusion

This study compared relational, document-oriented, and graph database systems for analytical queries on a large behavioral dataset. The experiments showed that PostgreSQL achieved the fastest performance for structured aggregation workloads, while MongoDB offered flexible ingestion of semi-structured data and Neo4j provided an intuitive model for relationship-based analysis. The results demonstrate that database choice should depend on the structure of the data and the nature of the analytical queries. In large-scale systems, a hybrid architecture combining relational, document, and graph databases may provide the most scalable solution.

Appendix A - Query Files Included in /scripts

The following analysis files were created during the conversation:

File	Purpose
q1.sql / q1.js / q1.cypher	Campaign effectiveness
q2.sql / q2.js / q2.cypher	Top recommended products
q3.sql / q3.js / q3.cypher	Category keyword search
clean_data.py	Dataset cleaning and preprocessing
load_data_psql.sql	PostgreSQL schema creation and data loading
load_data_mongodb.js	MongoDB loading
load_data_graph.cypher	Neo4j loading